

The following is a mix of information taken from different readme files that all explain different aspects of my 2019 iMac Ubuntu server running Docker containers and hosting an MQTT Broker, a network monitoring environment, a web server with test and prod sites on the server, plus a dev environment on my Mac Studio dev machine, and some other details. Please consolidate all of this information into one well organized readme file with an index at the top that links to the different topics in the document.

Monitoring Playbook — Grafana · LibreNMS · Prometheus (+ Docker via Telegraf)

> This document explains **where to find your dashboards**, **how they relate**, the **architecture** of your setup, and **operations** (CLI-driven panel/dashboard management and backups). It's tailored to the environment you've been building on `192.168.2.10`.

Table of Contents

- [1] Where to find dashboards (and how to view them)](#1-where-to-find-dashboards-and-how-to-view-them)
 - [Grafana](#grafana)
 - [LibreNMS](#librenms)
 - [Prometheus](#prometheus)
 - [MitchellNET Health Monitoring (Top-Level) — NEW](#mitchellnet-health-monitoring-top-level--new)
- [2] How they relate (mental model)](#2-how-they-relate-mental-model)
- [3] Architecture details](#3-architecture-details)
 - [3.1 Grafana](#31-grafana)
 - [3.2 LibreNMS](#32-librenms)
 - [3.3 Telegraf (Docker metrics)](#33-telegraf-docker-metrics)
 - [3.4 InfluxDB (v1.x HTTP interface on :8086)](#34-influxdb-v1x-http-interface-on-8086)
- [4] Operations — CLI-first](#4-operations--cli-first)
 - [4.1 Grafana API: add a templated variable + two panels (Docker CPU/Mem)](#41-grafana-api-add-a-templated-variable--two-panels-docker-cpumem)
 - [4.2 Verify/repair data flow](#42-verifyrepair-data-flow)
- [5] Backups & restoration](#5-backups--restoration)
 - [5.1 Grafana](#51-grafana)
 - [5.2 LibreNMS](#52-librenms)
 - [5.3 Prometheus](#53-prometheus)
 - [5.4 InfluxDB (v1.x)](#54-influxdb-v1x)
- [6] Adding panels/dashboards via CLI (patterns)](#6-adding-panelsdashboards-via-cli-patterns)
 - [6.1 Create or update a dashboard](#61-create-or-update-a-dashboard)
 - [6.2 Patch an existing dashboard (safe pattern)](#62-patch-an-existing-dashboard-safe-pattern)
- [7] What Docker monitoring is done now](#7-what-docker-monitoring-is-done-now)
- [8] Security notes](#8-security-notes)
- [9] Quick Troubleshooting](#9-quick-troubleshooting)

1) Where to find dashboards (and how to view them)

Grafana

- **URL (your host):** `http://192.168.2.10:3000`
- **Dashboards:**
 - **Ubuntu (LibreNMS) – Minimal** — UID: `ubuntu-librenms-min`
 - Navigate directly: `http://192.168.2.10:3000/d/ubuntu-librenms-min`
 - **MitchellNET Health Monitoring** — UID: `mitchellnet-health` ****(NEW)****
 - Navigate directly: `http://192.168.2.10:3000/d/mitchellnet-health`
 - **Internet Latency (Detail)** — UID: `net-latency-detail` ****(NEW)****
 - Navigate directly: `http://192.168.2.10:3000/d/net-latency-detail`
- **Datasources:**
 - **InfluxDB** (points to `http://192.168.2.10:8086`, DB: `librenms`) — used by LibreNMS & Telegraf metrics.
 - **Prometheus** (proxy mode; inside Docker it can be `http://prometheus:9090`, host-mapped `9090`) — used for exporter metrics (node-exporter, blackbox).
- **Using the dashboards**
 - **Time range:** Top-right time picker, e.g., last 6h.
 - **Auto-refresh:** Set e.g., `30s` for live charts.
 - **Templating (Container dropdown):** Choose one or more containers or **All** to display **per-container CPU/Memory** time series coming from Telegraf → InfluxDB.

> Tip: If you ever don't see the "Container" variable, refresh the page or ensure the Telegraf docker input is running and writing to InfluxDB (see Section 4.2).

LibreNMS

- **Purpose:** SNMP-based device monitoring (interfaces, mempools, CPU, sensors, etc.).
- **UI location:** Depends on your Docker port mapping or reverse proxy. To discover the exact port:

```
```bash
docker ps --format 'table {{.Names}} {{.Ports}}' | egrep -i 'librenms|nginx|httpd|apache'
Example: librenms exposes 8000->80/tcp => http://192.168.2.10:8000
```
```
- **What you'll see:** Devices list, graphs (RRD), alerts. LibreNMS also **writes** selected metrics to InfluxDB (e.g., `ports`, `processors`, `mempool` → stored as `mempool`).

Prometheus

- **Purpose:** Time-series database via scraping exporters (node_exporter, blackbox, etc.).
- **UI location:** Find the mapped port (commonly `9090`):

```
```bash
docker ps --format 'table {{.Names}} {{.Ports}}' | grep -i prometheus
Example: 9090/tcp -> http://192.168.2.10:9090
```
```
- **What you'll see:** Expression browser, targets, alerts (if configured), and instant or range queries. Grafana can query Prometheus too (your Grafana includes a Prometheus datasource).

MitchellNET Health Monitoring (Top-Level) — NEW

- **UID:** `mitchellnet-health`
- **Panels (today):**
 - **Internet Latency (google.com)** — **Stat** panel backed by Prometheus **blackbox_exporter** via the `blackbox\_http` job querying `probe\_duration\_seconds{job="blackbox\_http", instance="https://www.google.com/generate\_204"}`.
 - **Display:** Rolling average (1m) in **ms**; **green** < 200 ms, **yellow** 200–500 ms, **red** > 500 ms.
 - **Drilldown:** Panel link opens **Internet Latency (Detail)** (UID `net-latency-detail`) showing a full timeline.
 - **Ubuntu Server Status** — **Stat** panel using `up{job="node-exporter",instance="node-exporter:9100"}` (1 = UP, 0 = DOWN).
 - **Link:** Opens **Ubuntu (LibreNMS) – Minimal** for deep system graphs.
- **Notes / specifics for this environment:**
 - **Blackbox exporter** is published on the host at `192.168.2.10:9115`. The Prometheus job **relabels** the blackbox target to hit the exporter via that **host IP**, bypassing Docker DNS:

```
``yaml
- target_label: __address__
  replacement: 192.168.2.10:9115
``
```
 - **Prometheus** scrapes the blackbox HTTP module every **1m**, and the timeline dashboard uses `avg\_over\_time(...[ \$\_\_interval ])\*1000` to render in ms.

2) How they relate (mental model)

...

[Devices via SNMP] —► LibreNMS —► RRD (built-in)
└─► InfluxDB: librenms (measurements: ports, mempool, processors, app, ...)

[Docker Host] —► Telegraf (docker input) —► InfluxDB: librenms
(docker_container_cpu/mem/net/...)

[Exporters] —► Prometheus (scrapes on intervals) —► Prometheus TSDB (queried by Grafana)
└─► blackbox_exporter (HTTP probe to https://www.google.com/generate_204, relabeled to 192.168.2.10:9115)

Grafana —► InfluxDB (LibreNMS + Telegraf) ► Dashboards (Interface bps, Memory %, Docker per-container)
└─► Prometheus ► Dashboards (Internet Latency, Ubuntu Server status)

...

- **LibreNMS** polls devices, stores to **RRD**, and (when enabled) **mirrors** a subset into **InfluxDB** for easy Grafana queries.

- **Telegraf** collects **Docker per-container** metrics and writes them to **InfluxDB** (same `librenms`` database), allowing templated container dashboards.
- **Grafana** reads **InfluxDB** & **Prometheus** to present your dashboards.
- **Prometheus** is separate and used for exporters; Grafana's Prometheus datasource is already present but read-only by default.

3) Architecture details

3.1 Grafana

- **URL:** `http://192.168.2.10:3000``
- **Data sources:**
 - **InfluxDB** (proxy mode): `http://192.168.2.10:8086``, DB `librenms``, user `librenms`` (if configured), JSON data `dbName: "librenms"`.
 - **Prometheus**: inside Docker network it can be `http://prometheus:9090``; externally mapped `9090`` for host access. Grafana queries Prom via **proxy** access.
- **Authentication:** API tokens (Bearer). **Do not embed raw tokens** in source control.
- Create/rotate in Grafana UI:  → **Users & access** → **Service accounts / API keys**.
- **Environment variable usage** (recommended):


```
```bash
export GRAFANA_URL="http://192.168.2.10:3000"
export GRAFANA_TOKEN="REDACTED_USE_YOUR_TOKEN_HERE"
```
```
- **Dashboards of record:**
 - `Ubuntu (LibreNMS) – Minimal`` (`uid: ubuntu-librenms-min``)
 - `MitchellNET Health Monitoring`` (`uid: mitchellnet-health``) **(NEW)**
 - `Internet Latency (Detail)`` (`uid: net-latency-detail``) **(NEW)**
- **Representative Panels & Variables:**
 - **Interface bps (Ingress / Egress)** → Influx `ports`` (`ifInBits_rate``, `ifOutBits_rate``)
 - **Memory Used (%)** → Influx `mempool`` → compute %
 - **Docker CPU per container (%)** → Influx `docker_container_cpu`` (`usage_percent``)
 - **Docker Memory per container (%)** → Influx `docker_container_mem`` (`usage_percent``)
 - **\$container** (multi-select; **All** → `.*``) drives Docker templated queries
 - **Internet Latency (google.com)** → PromQL rolling average in ms with thresholds (green/yellow/red)
 - **Ubuntu Server Status** → `up{job="node-exporter",instance="node-exporter:9100"}`` (UP/DOWN) with link to Ubuntu dashboard

3.2 LibreNMS

- **Containerized**; polls via SNMP and updates RRDs; configured to write to **InfluxDB** (`librenms`` DB). Example checks:


```
```bash
docker exec -it librenms bash -lc 'php lnms config:get influxdb.enable'
docker exec -it librenms bash -lc 'php lnms config:get influxdb.host'
docker exec -it librenms bash -lc 'php lnms config:get influxdb.db'
```
```
- **Key Influx measurements observed**: `ports``, `processors``, `mempool`` (`mempool` stored under measurement `mempool``).

3.3 Telegraf (Docker metrics)

- ****Container run**** (minimal config):

```
```toml
[agent]
 interval = "10s"
 round_interval = true
 flush_interval = "10s"

[[outputs.influxdb]]
 urls = ["http://192.168.2.10:8086"]
 database = "librenms"
 skip_database_creation = true

[[inputs.docker]]
 endpoint = "unix:///var/run/docker.sock"
 gather_services = false
 container_name_include = []
 container_name_exclude = []
```
```

- ****Run**** (with Docker group pass-through):

```
```bash
SOCK_GID=$(stat -c '%g' /var/run/docker.sock)

docker rm -f telegraf 2>/dev/null || true
docker run -d --name telegraf --restart unless-stopped \
 --group-add "$SOCK_GID" \
 -v /var/run/docker.sock:/var/run/docker.sock:ro \
 -v "$PWD/telegraf.conf":/etc/telegraf/telegraf.conf:ro \
 telegraf:latest
```

# Verify measurements and tags exist:

```
curl -s "http://192.168.2.10:8086/query" \
 --data-urlencode "db=librenms" \
 --data-urlencode 'q=SHOW MEASUREMENTS WITH MEASUREMENT =~ /^docker_container_/\'
| jq -r '.results[0].series[0].values[][0]'
```

```
curl -s "http://192.168.2.10:8086/query" \
 --data-urlencode "db=librenms" \
 --data-urlencode 'q=SHOW TAG VALUES FROM "docker_container_cpu" WITH KEY =
"container_name"'
```
```

3.4 InfluxDB (v1.x HTTP interface on :8086)

- ****DB:**** `librenms`

- ****Smoke tests:****

```
```bash
curl -s "http://192.168.2.10:8086/ping" -i | head -n1 # Expect 204
curl -s "http://192.168.2.10:8086/query" --data-urlencode "q=SHOW DATABASES"
```

```

...
- **Key metrics & examples:**
 ``bash
 # Interface rates (mean over buckets)
 curl -s "http://192.168.2.10:8086/query" --data-urlencode "db=librenms" \
 --data-urlencode "q=SELECT mean(\"ifInBits_rate\") FROM ports WHERE
 \"hostname\"='192.168.2.10' AND \"ifName\"='enp4s0f0' AND time>now()-30m GROUP BY
 time(1m) LIMIT 3"

 # Memory % from mempool
 curl -s "http://192.168.2.10:8086/query" --data-urlencode "db=librenms" \
 --data-urlencode "q=SELECT (mean(\"used\")/(mean(\"used\")+mean(\"free\")))*100 FROM
 mempool WHERE \"hostname\"='192.168.2.10' AND \"mempool_class\"='system' AND
 time>now()-30m GROUP BY time(1m) LIMIT 3"
 ...

```

---

## ## 4) Operations — CLI-first

### 4.1 Grafana API: add a templated variable + two panels (Docker CPU/Mem)  
 > Uses your existing **\*\*InfluxDB\*\*** datasource and **\*\*dashboard UID\*\***.

```

``bash
----- env -----
export GRAFANA_URL="http://192.168.2.10:3000"
export GRAFANA_TOKEN="REDACTED_USE_YOUR_TOKEN_HERE"
export DASH_UID="ubuntu-librenms-min"
export DS_NAME="influxdb"

Resolve datasource UID robustly (list-all endpoint)
export DS_UID=$(curl -s -H "Authorization: Bearer $GRAFANA_TOKEN" \
 "$GRAFANA_URL/api/datasources" | jq -r --arg name "$DS_NAME" \
 '.[] | select(.type=="influxdb" and .name==$name) | .uid')

Fetch latest dashboard JSON
curl -s -H "Authorization: Bearer $GRAFANA_TOKEN" \
 "$GRAFANA_URL/api/dashboards/uid/$DASH_UID" > current.json

Build templated queries
qCPU=$(printf 'SELECT mean("usage_percent") FROM "docker_container_cpu" WHERE
("container_name" =~ /^$container$/) AND $timeFilter GROUP BY time($__interval) fill(previous)')
qMEM=$(printf 'SELECT mean("usage_percent") FROM "docker_container_mem" WHERE
("container_name" =~ /^$container$/) AND $timeFilter GROUP BY time($__interval) fill(previous)')

jq patch
cat > patch_docker.jq <<'JQ'
def ensureTemplating:
 if (.dashboard.templating? and .dashboard.templating.list?) then .

```

```

else (.dashboard.templating = { list: [] }) end;

def upsertContainerVar($ds_uid):
 .dashboard.templating.list |= (
 map(select(.name != "container"))
 + [
 {
 "type": "query",
 "name": "container",
 "label": "Container",
 "hide": 0,
 "datasource": { "type": "influxdb", "uid": $ds_uid },
 "query": "SHOW TAG VALUES FROM \"docker_container_cpu\" WITH KEY =
\"container_name\" WHERE time > now() - 24h",
 "refresh": 2,
 "sort": 1,
 "multi": true,
 "includeAll": true,
 "allValue": ".*",
 "regex": "",
 "current": null,
 "options": []
 }
]
);

def nextY:
 ([.dashboard.panels[]? | (.gridPos.y + .gridPos.h)] | max) // 0;

def appendPanels($ds_uid; $qCPU; $qMEM):
 .dashboard.panels += [
 {
 "type": "timeseries",
 "title": "Docker CPU per container (%)",
 "gridPos": { "h": 8, "w": 12, "x": 0, "y": (nextY)},
 "fieldConfig": { "defaults": { "unit": "percent", "min": 0, "max": 100 }, "overrides": [] },
 "datasource": { "type": "influxdb", "uid": $ds_uid },
 "targets": [{ "refId": "A", "rawQuery": true, "query": $qCPU }],
 "options": { "legend": { "displayMode": "list", "placement": "bottom" } }
 },
 {
 "type": "timeseries",
 "title": "Docker Memory per container (%)",
 "gridPos": { "h": 8, "w": 12, "x": 12, "y": (nextY)},
 "fieldConfig": { "defaults": { "unit": "percent", "min": 0, "max": 100 }, "overrides": [] },
 "datasource": { "type": "influxdb", "uid": $ds_uid },
 "targets": [{ "refId": "A", "rawQuery": true, "query": $qMEM }],
 "options": { "legend": { "displayMode": "list", "placement": "bottom" } }
 }
]

```

```

];

.dashboard.version = (.dashboard.version)
| ensureTemplating
| upsertContainerVar($ds_uid)
| appendPanels($ds_uid; $qCPU; $qMEM)
JQ

jq -f patch_docker.jq \
 --arg ds_uid "$DS_UID" \
 --arg qCPU "$qCPU" \
 --arg qMEM "$qMEM" \
 current.json > updated.json

curl -s -X POST "$GRAFANA_URL/api/dashboards/db" \
 -H "Authorization: Bearer $GRAFANA_TOKEN" \
 -H "Content-Type: application/json" \
 --data @updated.json | jq .

```

> After a refresh, your dashboard shows the **Container** dropdown and two panels.

---

#### ### 4.2 Verify/repair data flow

```

``bash
Telegraf running and has Docker socket access?
docker logs --tail 80 telegraf

Influx has docker_container_* measurements?
curl -s "http://192.168.2.10:8086/query" \
 --data-urlencode "db=librenms" \
 --data-urlencode 'q=SHOW MEASUREMENTS WITH MEASUREMENT =~ /^docker_container_/ |
jq .

LibreNMS → Influx mempool & ports:
curl -s "http://192.168.2.10:8086/query" \
 --data-urlencode "db=librenms" \
 --data-urlencode 'q=SHOW FIELD KEYS FROM "mempool"'
curl -s "http://192.168.2.10:8086/query" \
 --data-urlencode "db=librenms" \
 --data-urlencode 'q=SHOW FIELD KEYS FROM "ports"'

```

---

#### ## 5) Backups & restoration



### ### 5.1 Grafana

```
- **What to back up:**
- `/var/lib/grafana` (SQLite DB, user sessions, preferences)
- `/etc/grafana/grafana.ini` and `/etc/grafana/provisioning` (if used)
- **Dashboards & datasources via API** (export JSON for infra-as-code)
- **API export examples:**
 ``bash
 # Dashboard JSON
 curl -s -H "Authorization: Bearer $GRAFANA_TOKEN" \
 "$GRAFANA_URL/api/dashboards/uid/ubuntu-librenms-min" > grafana-dashboard-ubuntu-
 librenms-min.json

 # All datasources
 curl -s -H "Authorization: Bearer $GRAFANA_TOKEN" \
 "$GRAFANA_URL/api/datasources" > grafana-datasources.json
 ...

- **File-level backup (container):**
 ``bash
 docker run --rm --volumes-from grafana -v "$PWD:/backup" alpine \
 sh -c 'cd /var/lib && tar czf /backup/grafana-data.tgz grafana'
 ...
```

### ### 5.2 LibreNMS

```
- **Back up:**
- RRDs (usually mounted under `/data/rrd` in the container)
- Config (`/opt/librenms/config.php`), and DB (MariaDB container volume/snapshot if separate)
- **Example (RRD):**
 ``bash
 docker run --rm --volumes-from librenms -v "$PWD:/backup" alpine \
 sh -c 'cd /data && tar czf /backup/librenms-rrd.tgz rrd'
 ...
```

### ### 5.3 Prometheus

```
- **Snapshot the TSDB via API:**
 ``bash
 # Make a snapshot; returns path under /prometheus/snapshots/<id>
 curl -XPOST "http://192.168.2.10:9090/api/v1/admin/tsdb/snapshot?skip_head=true"

 # Then copy the snapshot dir from the container
 docker cp prometheus:/prometheus/snapshots ./prometheus-snapshots
 ...
```

### ### 5.4 InfluxDB (v1.x)

```
- **Portable backup from container:**
 ``bash
 # If running official influxdb 1.x image:
 docker exec influxdb influxd backup -portable /backup
 docker cp influxdb:/backup ./influxdb-backup
 ...
```

- \*\*File-level copy (if not using the CLI):\*\*

```
```bash
docker run --rm --volumes-from influxdb -v "$PWD:/backup" alpine \
  sh -c 'cd /var/lib && tar czf /backup/influxdb-data.tgz influxdb'
```
```

> Automate backups via cron on the host; keep secrets/tokens out of the tarballs when possible.

---

## ## 6) Adding panels/dashboards via CLI (patterns)

### ### 6.1 Create or update a dashboard

```
```bash
# Create minimal dashboard with a single panel (InfluxDB query)
cat > dash.json <<'JSON'
{
  "dashboard": {
    "id": null,
    "uid": "example-min",
    "title": "Example – Minimal",
    "schemaVersion": 38,
    "version": 1,
    "refresh": "30s",
    "panels": [
      {
        "type": "timeseries",
        "title": "Interface In/Out bps",
        "gridPos": {"h": 8, "w": 24, "x": 0, "y": 0},
        "fieldConfig": {"defaults": {"unit": "bps"}, "overrides": []},
        "datasource": {"type": "influxdb", "uid": "REPLACE_DS_UID"},
        "targets": [
          {"refId": "A", "rawQuery": true, "query": "SELECT mean(\"ifInBits_rate\") FROM \"ports\" WHERE (\"hostname\"='192.168.2.10' AND \"ifName\"='enp4s0f0') AND $timeFilter GROUP BY time($__interval) fill(previous)"},
          {"refId": "B", "rawQuery": true, "query": "SELECT mean(\"ifOutBits_rate\") FROM \"ports\" WHERE (\"hostname\"='192.168.2.10' AND \"ifName\"='enp4s0f0') AND $timeFilter GROUP BY time($__interval) fill(previous)"}
        ]
      }
    ],
    "overwrite": true
  }
}
JSON
```

Replace and POST

```
DS_UID=$(curl -s -H "Authorization: Bearer $GRAFANA_TOKEN" \
"$GRAFANA_URL/api/datasources" | jq -r '.[] | select(.type=="influxdb") | .uid' | head -n1)
```

```
sed -i "s/REPLACE_DS_UID/$DS_UID/g" dash.json
```

```
curl -s -X POST "$GRAFANA_URL/api/dashboards/db" \  
-H "Authorization: Bearer $GRAFANA_TOKEN" \  
-H "Content-Type: application/json" \  
--data @dash.json | jq .  
...
```

6.2 Patch an existing dashboard (safe pattern)

```
```bash  
curl -s -H "Authorization: Bearer $GRAFANA_TOKEN" \
"$GRAFANA_URL/api/dashboards/uid/$DASH_UID" > current.json
```

# ...prepare a jq patch (as shown earlier)...

```
jq -f patch.jq current.json > updated.json
```

```
curl -s -X POST "$GRAFANA_URL/api/dashboards/db" \
-H "Authorization: Bearer $GRAFANA_TOKEN" \
-H "Content-Type: application/json" \
--data @updated.json | jq .
...
```

---

## 7) What Docker monitoring is done now

- **CPU per container:** ``docker_container_cpu` → field `usage_percent``
- **Memory per container:** ``docker_container_mem` → field `usage_percent` (also raw `usage` & `limit`)`
- **Available for expansion:** ``docker_container_net` (rx/tx bytes), `docker_container_blkio` (I/O), `docker_container_status/health``
- **Grafana variable:** ``$container` filters charts to one or more container names.`

---

## 8) Security notes

- Treat **Grafana tokens** as secrets. Prefer **service accounts** with minimal scope and rotate regularly.
- Avoid committing tokens into repositories; use environment variables or a `.env` file with restricted ACLs.`
- For backups, **exclude** tokens and config files that carry secrets unless you encrypt the archives.

---

## 9) Quick Troubleshooting

- **“No data” in Grafana panel**

- Confirm the measurement & field exist with Influx queries (see 3.4).
- Check time range and `fill(previous)` behavior.
- Verify the datasource `uid` is correct inside the panel JSON.
- **Telegraf docker input errors**
  - Ensure group access to `/var/run/docker.sock` (`--group-add \$(stat -c '%g' /var/run/docker.sock)`).
  - Confirm the config does not include unsupported options for your Telegraf version.
- **Grafana API 401**
  - Token wrong/expired/insufficient scope. Re-issue a token and export as `GRAFANA\_TOKEN`.

---

\*End of document.\*

## # Monitoring Playbook — Grafana · LibreNMS · Prometheus (+ Docker via Telegraf)

> This document explains **where to find your dashboards**, **how they relate**, the **architecture** of your setup, and **operations** (CLI-driven panel/dashboard management and backups). It's tailored to the environment you've been building on `192.168.2.10`.

---

### ## 1) Where to find dashboards (and how to view them)

#### ### Grafana

- **URL (your host):** `http://192.168.2.10:3000`
- **Dashboards:**
  - **Ubuntu (LibreNMS) – Minimal** — UID: `ubuntu-librenms-min`
  - Navigate directly: `http://192.168.2.10:3000/d/ubuntu-librenms-min`
- **Datasources:**
  - **InfluxDB** (points to `http://192.168.2.10:8086`, DB: `librenms`) — used by LibreNMS & Telegraf metrics.
  - **Prometheus** (points to `http://prometheus:9090` inside Docker) — used for any Prometheus-scraped metrics.
- **Using the dashboard**
  - **Time range:** Top-right time picker, e.g., last 6h.
  - **Auto-refresh:** Set e.g., `30s` for live charts.
  - **Templating (Container dropdown):** Choose one or more containers or **All** to display **per-container CPU/Memory** time series coming from Telegraf → InfluxDB.

> Tip: If you ever don't see the "Container" variable, refresh the page or ensure the Telegraf docker input is running and writing to InfluxDB (see Section 4.2).

---

#### ### LibreNMS

- **Purpose:** SNMP-based device monitoring (interfaces, mem pools, CPU, sensors, etc.).
- **UI location:** Depends on your Docker port mapping or reverse proxy. To discover the exact port:
 

```
bash
docker ps --format 'table {{.Names}} {{.Ports}}' | egrep -i 'librenms|nginx|httpd|apache'
```

# Example: librenms exposes 8000->80/tcp => http://192.168.2.10:8000

...

- **What you'll see:** Devices list, graphs (RRD), alerts. LibreNMS also **writes** selected metrics to InfluxDB (e.g., `ports`, `processors`, `mempool` → stored as `mempool`).

---

### ### Prometheus

- **Purpose:** Time-series database via scraping exporters (node\_exporter, blackbox, etc.).

- **UI location:** Find the mapped port (commonly `9090`):

```bash

docker ps --format 'table {{.Names}} {{.Ports}}' | grep -i prometheus

Example: 9090/tcp -> http://192.168.2.10:9090

...

- **What you'll see:** Expression browser, targets, alerts (if configured), and instant or range queries. Grafana can query Prometheus too (your Grafana includes a Prometheus datasource).

2) How they relate (mental model)

...

[Devices via SNMP] —► LibreNMS —► RRD (built-in)
└─► InfluxDB: librenms (measurements: ports, mempool, processors, app, ...)

[Docker Host] —► Telegraf (docker input) —► InfluxDB: librenms
(docker_container_cpu/mem/net/...)

[Exporters] —► Prometheus (scrapes on intervals) —► Prometheus TSDB (queried by Grafana)

Grafana —► InfluxDB (LibreNMS + Telegraf) ► Dashboards (Interface bps, Memory %, Docker per-container)
└─► Prometheus ► Dashboards (exporter metrics, etc.)

...

- **LibreNMS** polls devices, stores to **RRD**, and (when enabled) **mirrors** a subset into **InfluxDB** for easy Grafana queries.

- **Telegraf** collects **Docker per-container** metrics and writes them to **InfluxDB** (same `librenms` database), allowing templated container dashboards.

- **Grafana** reads **InfluxDB** & **Prometheus** to present your dashboards.

- **Prometheus** is separate and used for exporters; Grafana's Prometheus datasource is already present but read-only by default.

3) Architecture details

3.1 Grafana

- **URL:** `http://192.168.2.10:3000`

- **Data sources:**
 - **InfluxDB** (proxy mode): `http://192.168.2.10:8086`, DB `librenms`, user `librenms` (if configured), JSON data `dbName: "librenms"`.
 - **Prometheus**: typically `http://prometheus:9090` inside Docker network; externally map a host port (e.g., `9090:9090`).
 - **Authentication:** API tokens (Bearer). **Do not embed raw tokens** in source control.
 - Create/rotate in Grafana UI:  → **Users & access** → **Service accounts / API keys**.
 - **Environment variable usage** (recommended):

```

bash
export GRAFANA_URL="http://192.168.2.10:3000"
export GRAFANA_TOKEN="REDACTED_USE_YOUR_TOKEN_HERE"

```
- **Dashboard of record:** `Ubuntu (LibreNMS) – Minimal` (`uid: ubuntu-librenms-min`)
- **Panels (examples):**
 - **Interface bps (Ingress / Egress)** → Influx `ports` (`ifInBits_rate`, `ifOutBits_rate`)
 - **Memory Used (%)** → Influx `mempool` (`used` and `free` into %)
 - **Docker CPU per container (%)** → Influx `docker_container_cpu` (`usage_percent`)
 - **Docker Memory per container (%)** → Influx `docker_container_mem` (`usage_percent`)
- **Variables:**
 - **`$container`** (multi-select w/ All → `.*`) → drives templated queries for Docker panels

3.2 LibreNMS

- **Containerized**; polls via SNMP and updates RRDs; configured to write to **InfluxDB** (`librenms` DB). Example checks:

```

bash
docker exec -it librenms bash -lc 'php lnms config:get influxdb.enable'
docker exec -it librenms bash -lc 'php lnms config:get influxdb.host'
docker exec -it librenms bash -lc 'php lnms config:get influxdb.db'

```

- **Key Influx measurements observed**: `ports`, `processors`, `mempool` (`mempool` stored under measurement `mempool`).

3.3 Telegraf (Docker metrics)

- **Container run** (minimal config):

```

toml
[agent]
interval = "10s"
round_interval = true
flush_interval = "10s"

[[outputs.influxdb]]
urls = ["http://192.168.2.10:8086"]
database = "librenms"
skip_database_creation = true

```

```

[[inputs.docker]]
endpoint = "unix:///var/run/docker.sock"
gather_services = false
container_name_include = []

```

```

    container_name_exclude = []
    ...

- **Run** (with Docker group pass-through):
    ``bash
    SOCK_GID=$(stat -c '%g' /var/run/docker.sock)

    docker rm -f telegraf 2>/dev/null || true
    docker run -d --name telegraf --restart unless-stopped --group-add "$SOCK_GID" -v
/var/run/docker.sock:/var/run/docker.sock:ro -v "$PWD/telegraf.conf":/etc/telegraf/telegraf.conf:ro
telegraf:latest

```

```

# Verify measurements and tags exist:
curl -s "http://192.168.2.10:8086/query" --data-urlencode "db=librenms" --data-urlencode
'q=SHOW MEASUREMENTS WITH MEASUREMENT =~ /^docker_container_/ | jq -r
'.results[0].series[0].values[][0]'

```

```

curl -s "http://192.168.2.10:8086/query" --data-urlencode "db=librenms" --data-urlencode
'q=SHOW TAG VALUES FROM "docker_container_cpu" WITH KEY = "container_name"'
...

```

3.4 InfluxDB (v1.x HTTP interface on :8086)

```

- **DB:** `librenms`
- **Smoke tests:**
    ``bash
    curl -s "http://192.168.2.10:8086/ping" -i | head -n1 # Expect 204
    curl -s "http://192.168.2.10:8086/query" --data-urlencode "q=SHOW DATABASES"
    ...

```

```

- **Key metrics & examples:**
    ``bash
    # Interface rates (mean over buckets)
    curl -s "http://192.168.2.10:8086/query" --data-urlencode "db=librenms" --data-urlencode
"q=SELECT mean(\"ifInBits_rate\") FROM ports WHERE \"hostname\"='192.168.2.10' AND
\"ifName\"='enp4s0f0' AND time>now()-30m GROUP BY time(1m) LIMIT 3"

```

```

# Memory % from mempool
curl -s "http://192.168.2.10:8086/query" --data-urlencode "db=librenms" --data-urlencode
"q=SELECT (mean(\"used\")/(mean(\"used\")+mean(\"free\")))*100 FROM mempool WHERE
\"hostname\"='192.168.2.10' AND \"mempool_class\"='system' AND time>now()-30m GROUP BY
time(1m) LIMIT 3"
...

```

4) Operations — CLI-first

4.1 Grafana API: add a templated variable + two panels (Docker CPU/Mem)

> Uses your existing ****InfluxDB**** datasource and ****dashboard UID****.

```

``bash

```

```

# ----- env -----
export GRAFANA_URL="http://192.168.2.10:3000"
export GRAFANA_TOKEN="REDACTED_USE_YOUR_TOKEN_HERE"
export DASH_UID="ubuntu-librenms-min"
export DS_NAME="influxdb"

# Resolve datasource UID robustly (list-all endpoint)
export DS_UID=$(curl -s -H "Authorization: Bearer $GRAFANA_TOKEN"
"$GRAFANA_URL/api/datasources" | jq -r --arg name "$DS_NAME" '[] | select(.type=="influxdb"
and .name==$name) | .uid')

# Fetch latest dashboard JSON
curl -s -H "Authorization: Bearer $GRAFANA_TOKEN"
"$GRAFANA_URL/api/dashboards/uid/$DASH_UID" > current.json

# Build templated queries
qCPU=$(printf 'SELECT mean("usage_percent") FROM "docker_container_cpu" WHERE
("container_name" =~ /^$container$/) AND $timeFilter GROUP BY time($__interval) fill(previous)')
qMEM=$(printf 'SELECT mean("usage_percent") FROM "docker_container_mem" WHERE
("container_name" =~ /^$container$/) AND $timeFilter GROUP BY time($__interval) fill(previous)')

# jq patch
cat > patch_docker.jq <<'JQ'
def ensureTemplating:
  if (.dashboard.templating? and .dashboard.templating.list?) then .
  else (.dashboard.templating = { list: [] }) end;

def upsertContainerVar($ds_uid):
  .dashboard.templating.list |= (
    map(select(.name != "container"))
    + [
      {
        "type": "query",
        "name": "container",
        "label": "Container",
        "hide": 0,
        "datasource": { "type": "influxdb", "uid": $ds_uid },
        "query": "SHOW TAG VALUES FROM "docker_container_cpu" WITH KEY =
"container_name" WHERE time > now() - 24h",
        "refresh": 2,
        "sort": 1,
        "multi": true,
        "includeAll": true,
        "allValue": ".*",
        "regex": "",
        "current": null,
        "options": []
      }
    ]
  )

```



```

);

def nextY:
  ( [ .dashboard.panels[]? | (.gridPos.y + .gridPos.h) ] | max ) // 0;

def appendPanels($ds_uid; $qCPU; $qMEM):
  .dashboard.panels += [
    {
      "type": "timeseries",
      "title": "Docker CPU per container (%)",
      "gridPos": { "h": 8, "w": 12, "x": 0, "y": (nextY)},
      "fieldConfig": { "defaults": { "unit": "percent", "min": 0, "max": 100 }, "overrides": [] },
      "datasource": { "type": "influxdb", "uid": $ds_uid},
      "targets": [ { "refId": "A", "rawQuery": true, "query": $qCPU } ],
      "options": { "legend": { "displayMode": "list", "placement": "bottom" } }
    },
    {
      "type": "timeseries",
      "title": "Docker Memory per container (%)",
      "gridPos": { "h": 8, "w": 12, "x": 12, "y": (nextY)},
      "fieldConfig": { "defaults": { "unit": "percent", "min": 0, "max": 100 }, "overrides": [] },
      "datasource": { "type": "influxdb", "uid": $ds_uid},
      "targets": [ { "refId": "A", "rawQuery": true, "query": $qMEM } ],
      "options": { "legend": { "displayMode": "list", "placement": "bottom" } }
    }
  ];

.dashboard.version = (.dashboard.version)
| ensureTemplating
| upsertContainerVar($ds_uid)
| appendPanels($ds_uid; $qCPU; $qMEM)
JQ

jq -f patch_docker.jq --arg ds_uid "$DS_UID" --arg qCPU "$qCPU" --arg qMEM "$qMEM"
current.json > updated.json

curl -s -X POST "$GRAFANA_URL/api/dashboards/db" -H "Authorization: Bearer
$GRAFANA_TOKEN" -H "Content-Type: application/json" --data @updated.json | jq .
```

```

> After a refresh, your dashboard shows the **Container** dropdown and two panels.

---

#### ### 4.2 Verify/repair data flow

```

```bash
# Telegraf running and has Docker socket access?
docker logs --tail 80 telegraf

```

```
# Influx has docker_container_* measurements?
curl -s "http://192.168.2.10:8086/query" --data-urlencode "db=librenms" --data-urlencode 'q=SHOW
MEASUREMENTS WITH MEASUREMENT =~ /^docker_container_/ | jq .
```

```
# LibreNMS → Influx mempool & ports:
```

```
curl -s "http://192.168.2.10:8086/query" --data-urlencode "db=librenms" --data-urlencode 'q=SHOW
FIELD KEYS FROM "mempool"'
curl -s "http://192.168.2.10:8086/query" --data-urlencode "db=librenms" --data-urlencode 'q=SHOW
FIELD KEYS FROM "ports"'
---
```

5) Backups & restoration

5.1 Grafana

```
- **What to back up:**
```

- `/var/lib/grafana` (SQLite DB, user sessions, preferences)`
- `/etc/grafana/grafana.ini` and /etc/grafana/provisioning` (if used)`
- **Dashboards & datasources via API** (export JSON for infra-as-code)

```
- **API export examples:**
```

```
``bash
```

```
# Dashboard JSON
```

```
curl -s -H "Authorization: Bearer $GRAFANA_TOKEN"
"$GRAFANA_URL/api/dashboards/uid/ubuntu-librenms-min" > grafana-dashboard-ubuntu-librenms-
min.json
```

```
# All datasources
```

```
curl -s -H "Authorization: Bearer $GRAFANA_TOKEN" "$GRAFANA_URL/api/datasources" >
grafana-datasources.json
---
```

```
- **File-level backup (container):**
```

```
``bash
```

```
docker run --rm --volumes-from grafana -v "$PWD:/backup" alpine sh -c 'cd /var/lib && tar czf
/backup/grafana-data.tgz grafana'
---
```

5.2 LibreNMS

```
- **Back up:**
```

- RRDs (usually mounted under `/data/rrd` in the container)`
- Config (`/opt/librenms/config.php`), and DB (MariaDB container volume/snapshot if separate)`

```
- **Example (RRD):**
```

```
``bash
```

```
docker run --rm --volumes-from librenms -v "$PWD:/backup" alpine sh -c 'cd /data && tar czf
/backup/librenms-rrd.tgz rrd'
---
```

5.3 Prometheus

- ****Snapshot the TSDB via API:****

```
```bash
Make a snapshot; returns path under /prometheus/snapshots/<id>
curl -XPOST "http://192.168.2.10:9090/api/v1/admin/tsdb/snapshot?skip_head=true"
```

```
Then copy the snapshot dir from the container
docker cp prometheus:/prometheus/snapshots ./prometheus-snapshots
```
```

5.4 InfluxDB (v1.x)

- ****Portable backup from container:****

```
```bash
If running official influxdb 1.x image:
docker exec influxdb influxd backup -portable /backup
docker cp influxdb:/backup ./influxdb-backup
```
```

- ****File-level copy (if not using the CLI):****

```
```bash
docker run --rm --volumes-from influxdb -v "$PWD:/backup" alpine sh -c 'cd /var/lib && tar czf
/backup/influxdb-data.tgz influxdb'
```
```

> Automate backups via cron on the host; keep secrets/tokens out of the tarballs when possible.

6) Adding panels/dashboards via CLI (patterns)

6.1 Create or update a dashboard

```
```bash
Create minimal dashboard with a single panel (InfluxDB query)
cat > dash.json <<'JSON'
{
 "dashboard": {
 "id": null,
 "uid": "example-min",
 "title": "Example – Minimal",
 "schemaVersion": 38,
 "version": 1,
 "refresh": "30s",
 "panels": [
 {
 "type": "timeseries",
 "title": "Interface In/Out bps",
 "gridPos": {"h": 8, "w": 24, "x": 0, "y": 0},
 "fieldConfig": {"defaults": {"unit": "bps"}, "overrides": []},
 "datasource": {"type": "influxdb", "uid": "REPLACE_DS_UID"},
 "targets": [
```

```

 {"refId":"A","rawQuery":true,"query":"SELECT mean(`ifInBits_rate`) FROM `ports`"
WHERE (`hostname`='192.168.2.10' AND `ifName`='enp4s0f0') AND $timeFilter GROUP BY
time($__interval) fill(previous)"},
 {"refId":"B","rawQuery":true,"query":"SELECT mean(`ifOutBits_rate`) FROM `ports`"
WHERE (`hostname`='192.168.2.10' AND `ifName`='enp4s0f0') AND $timeFilter GROUP BY
time($__interval) fill(previous)}
]
}
],
"overwrite": true
}
JSON

```

# Replace and POST

```

DS_UID=$(curl -s -H "Authorization: Bearer $GRAFANA_TOKEN"
"$GRAFANA_URL/api/datasources" | jq -r '[] | select(.type=="influxdb") | .uid' | head -n1)

```

```

sed -i "s/REPLACE_DS_UID/$DS_UID/g" dash.json

```

```

curl -s -X POST "$GRAFANA_URL/api/dashboards/db" -H "Authorization: Bearer
$GRAFANA_TOKEN" -H "Content-Type: application/json" --data @dash.json | jq .

```

### 6.2 Patch an existing dashboard (safe pattern)

```

```bash
curl -s -H "Authorization: Bearer $GRAFANA_TOKEN"
"$GRAFANA_URL/api/dashboards/uid/$DASH_UID" > current.json

```

...prepare a jq patch (as shown earlier)...

```

jq -f patch.jq current.json > updated.json

```

```

curl -s -X POST "$GRAFANA_URL/api/dashboards/db" -H "Authorization: Bearer
$GRAFANA_TOKEN" -H "Content-Type: application/json" --data @updated.json | jq .

```

7) What Docker monitoring is done now

- **CPU per container:** `docker\_container\_cpu` → field `usage\_percent`
- **Memory per container:** `docker\_container\_mem` → field `usage\_percent` (also raw `usage` & `limit`)
- **Available for expansion:** `docker\_container\_net` (rx/tx bytes), `docker\_container\_blkio` (I/O), `docker\_container\_status/health`
- **Grafana variable:** `container` filters charts to one or more container names.

8) Security notes

- Treat **Grafana tokens** as secrets. Prefer **service accounts** with minimal scope and rotate regularly.
- Avoid committing tokens into repositories; use environment variables or a `.env` file with restricted ACLs.
- For backups, **exclude** tokens and config files that carry secrets unless you encrypt the archives.

9) Quick Troubleshooting

- **“No data” in Grafana panel**
 - Confirm the measurement & field exist with Influx queries (see 3.4).
 - Check time range and `fill(previous)` behavior.
 - Verify the datasource `uid` is correct inside the panel JSON.
- **Telegraf docker input errors**
 - Ensure group access to `/var/run/docker.sock` (`--group-add $(stat -c '%g' /var/run/docker.sock)`).
 - Confirm the config does not include unsupported options for your Telegraf version.
- **Grafana API 401**
 - Token wrong/expired/insufficient scope. Re-issue a token and export as `GRAFANA_TOKEN`.

End of document.

MitchellNET (Internal WebServer)

This repository describes the full setup, configuration, and workflow of the **MitchellNET Internal WebServer**, a local, containerized NGINX-based deployment hosted on a 2019 iMac Ubuntu Server, with development and CI/CD orchestration managed via a Mac Studio and GitHub Actions.

You can read about the network monitoring system [\[here\]](#)(`dashboard.md`).

Table of Contents

1. [\[Overview\]](#)(#overview)
2. [\[System Architecture\]](#)(#system-architecture)
 - [\[Development Environment \(Mac Studio\)\]](#)(#development-environment-mac-studio)
 - [\[Version Control and GitHub Actions\]](#)(#version-control-and-github-actions)
 - [\[Server Environment \(2019 iMac Ubuntu\)\]](#)(#server-environment-2019-imac-ubuntu)
3. [\[Initial Setup\]](#)(#initial-setup)
 - [\[1. Local Development Setup\]](#)(#1-local-development-setup)
 - [\[2. GitHub Repository Setup\]](#)(#2-github-repository-setup)
 - [\[3. Ubuntu Server Setup\]](#)(#3-ubuntu-server-setup)
 - [\[4. NGINX Reverse Proxy and SSL Configuration\]](#)(#4-nginx-reverse-proxy-and-ssl-configuration)
4. [\[Deployment Workflow\]](#)(#deployment-workflow)
 - [\[Development → Test\]](#)(#development--test)

- [Test → Production Promotion](#test--production-promotion)
- 5. [Branching Strategy](#branching-strategy)
- 6. [Cloning and Replicating the Environment](#cloning-and-replicating-the-environment)
- 7. [Troubleshooting and Maintenance](#troubleshooting-and-maintenance)
- 8. [Dev quick reference](devQuickReference.md)
- 9. [Newer config info](README-New.md)

🧭 Overview

The MitchellNET web system provides an internal HTTPS website for local use, served by **Nginx** running in Docker containers on a **2019 iMac (Ubuntu Server 24.04.2 LTS)**. Development occurs on a **Mac Studio**, with automated CI/CD deployments using **GitHub Actions**.

Environments:

- **Development** → Mac Studio (local Visual Studio Code)
- **Test** → iMac Ubuntu Server via GitHub Actions (self-hosted runner)
- **Production** → iMac Ubuntu Server (promoted from Test)

Developer Notes:

If you wish to contribute to the MitchellNET internal website you can read developer notes [here] (DeveloperNotes.md).

🏗️ System Architecture

Development Environment (Mac Studio)

| Component | Details |

|-----|-----|

| **Hardware** | Apple Mac Studio |

| **OS** | macOS Sonoma |

| **IDE** | Visual Studio Code |

| **Git Client** | Built-in Git integration |

| **SSH Keys** | Stored in `~/.ssh/id\_rsa` and registered in GitHub Secrets |

| **Primary Directory** |

`/Users/andrewmitchell/Documents/visualStudioCode/html/projects/InternalWebServer` |

All source content and workflows are developed and pushed to the remote GitHub repository:

...

<https://github.com/theAgingApprentice/InternalWebServer.git>

...

Version Control and GitHub Actions

| Component | Details |

```

|-----|-----|
| **Repo** | `theAgingApprentice/InternalWebServer` |
| **Remote URL** | `https://github.com/theAgingApprentice/InternalWebServer.git` |
| **Runner** | Self-hosted runner on iMac (`imac-server-runner`) |
| **Secrets Used** | `SSH_PRIVATE_KEY`, `DEPLOY_USER`, `DEPLOY_HOST`,
`DEPLOY_PATH` |
| **Workflows** | `.github/workflows/deploy-test.yml`, `.github/workflows/deploy-prod.yml` |

```

These workflows automate:

- Deployment of `develop` branch → Test environment
- Deployment of `main` branch → Production environment

Server Environment (2019 iMac Ubuntu)

```

| Component | Details |
|-----|-----|
| **Hardware** | iMac 2019 (Model: iMac19,1) |
| **CPU** | 3.1 GHz 6-Core Intel Core i5 |
| **Memory** | 32 GB DDR4 |
| **OS** | Ubuntu Server 24.04.2 LTS |
| **Docker** | v28.3.1 |
| **NGINX** | `nginx:latest` Docker image |
| **Location** | Electronics Lab — accessible only on MitchellNET LAN |
| **IP Address** | `192.168.2.10` |

```

Directory Structure

```

...
/home/andrew/web_server/html/
├── index.html
├── dev/
│   └── index.html
├── test/
│   └── index.html
└── prod/
    └── index.html
...

```

🌀 Initial Setup

1. Local Development Setup

1. Clone the GitHub repository:

```

```bash
git clone https://github.com/theAgingApprentice/InternalWebServer.git
```

```

2. Create an SSH key if not already present:

```

```bash

```

```
ssh-keygen -t rsa -b 4096 -C "andrewmitchell@macstudio"
```

3. Add your public key to the Ubuntu server:

```
ssh-copy-id andrew@192.168.2.10
```

4. Test SSH connection:

```
ssh andrew@192.168.2.10
```

---

### ### 2. GitHub Repository Setup

1. Add a remote:

```
git remote add origin https://github.com/theAgingApprentice/InternalWebServer.git
```

2. Add repository secrets:

Name	Description
SSH_PRIVATE_KEY	Your Mac Studio's private SSH key
DEPLOY_USER	andrew
DEPLOY_HOST	192.168.2.10
DEPLOY_PATH	/home/andrew/web_server/html

3. Push branches:

```
git push origin develop
git push origin main
```

---

### ### 3. Ubuntu Server Setup

1. Install Docker and GitHub Runner:

```
sudo apt update && sudo apt install -y docker.io git
```

2. Register the GitHub self-hosted runner:

- Go to your repo → \*\*Settings → Actions → Runners → New self-hosted runner\*\*
- Follow the registration commands (as `andrew` user)

3. Start the runner service:

```
./svc.sh install
./svc.sh start
```

---



## ### 4. NGINX Reverse Proxy and SSL Configuration

### #### Proxy Overview

Three NGINX containers manage traffic:

Container	Purpose	Path
----- ----- -----		
`nginx-proxy`	Reverse proxy (SSL termination)	`/usr/share/nginx/html`
`nginx-test`	Serves test environment	`/home/andrew/web_server/html/test`
`nginx-prod`	Serves production environment	`/home/andrew/web_server/html/prod`

### #### SSL Certificate

Self-signed SSL/TLS cert:

```
```bash
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/private/selfsigned.key
-out /etc/ssl/certs/selfsigned.crt -subj
"/C=CA/ST=Ontario/L=London/O=TheAgingApprentice/CN=Andrew
Mitchell/emailAddress=va3wam@gmail.com"
```
```

---

## ## 🚀 Deployment Workflow

### #### Development → Test

When code is pushed to the `develop` branch:

1. GitHub Action (`deploy-test.yml`) runs on `imac-server-runner`.
2. Workflow connects to Ubuntu via SSH using secrets.
3. Files sync to `/home/andrew/web\_server/html/test/`.
4. `https://192.168.2.10/test/` serves the updated version.

### #### Test → Production Promotion

Once validated, changes are merged into `main`:

1. GitHub Action (`deploy-prod.yml`) runs.
2. Files deploy to `/home/andrew/web\_server/html/prod/`.
3. `https://192.168.2.10/` serves the production content.

---

## ## 🌿 Branching Strategy

| Branch            | Purpose                        | Deployment Target    |
|-------------------|--------------------------------|----------------------|
| ----- ----- ----- |                                |                      |
| `develop`         | Active development and testing | `/test/` environment |
| `main`            | Stable production-ready code   | `/prod/` environment |

**\*\*Workflow:\*\***

1. Create feature branches from `develop`
2. Open PRs into `develop` for integration testing

3. Merge `develop` → `main` for release to production

---

## ## 🔄 Cloning and Replicating the Environment

To recreate this setup on a new network or device:

1. Clone the repository:

```
```bash
git clone https://github.com/theAgingApprentice/InternalWebServer.git
```
```

2. Run the provided setup script (`setup\_environment.sh`) to:

- Generate SSH keys
- Register a new GitHub runner
- Deploy Docker + NGINX + certificates

3. Update `.github/workflows` secrets for the new environment.

---

## ## 🛠 Troubleshooting and Maintenance

- **Workflow Fails with SSH Error** → Ensure Ubuntu server is reachable by runner (private LAN only)

- **Certificate Mismatch** → Regenerate `selfsigned.crt` and `selfsigned.key`

- **Nginx Proxy Issues** → Check container logs:

```
```bash
sudo docker logs nginx-proxy
```
```

- **Re-register GitHub Runner** (if it goes offline):

```
```bash
./config.sh remove
./config.sh
```
```

---

© 2025 TheAgingApprentice — Internal use only.

## # NGINX Reverse Proxy — Production & Test Subdomain Setup

> **Access URLs**

>

> - **Production site**: [ `https://prod.mitchellnet.local` ](https://prod.mitchellnet.local)

> - **Test site**: [ `https://test.mitchellnet.local` ](https://test.mitchellnet.local)

> - **Bare IP**: [ `https://192.168.2.10` ](https://192.168.2.10) → redirects to `prod.mitchellnet.local`

>

> To resolve these names locally, ensure your `/etc/hosts` file contains:

> ```

> 192.168.2.10 prod.mitchellnet.local test.mitchellnet.local

> ```

---

## ## Overview

This repository documents the working **NGINX reverse-proxy** and **multi-environment web server** stack deployed on the `MitchellNET` internal network (`192.168.2.10`). The proxy container fronts two backend NGINX servers (`nginx-prod`` and `nginx-test``), each serving their own content directories and environment banners.

The setup runs on **Ubuntu Server 24.04.2 LTS** hosted on a **2019 iMac (model iMac19,1)** and uses Docker Compose for orchestration.

---

## ## 🧩 Architecture Summary

| Component               | Purpose                      | Host Path                                   | Container Mount                               |
|-------------------------|------------------------------|---------------------------------------------|-----------------------------------------------|
| ----- ----- ----- ----- |                              |                                             |                                               |
| <b>nginx-proxy</b>      | Reverse proxy for subdomains | <code>./nginx/conf.d`</code>                | <code>/etc/nginx/conf.d`</code>               |
| <b>nginx-prod</b>       | Production web root          | <code>./html/prod`</code>                   | <code>/usr/share/nginx/html`</code>           |
| <b>nginx-test</b>       | Test web root                | <code>./html/test`</code>                   | <code>/usr/share/nginx/html`</code>           |
| <b>Self-signed SSL</b>  | TLS cert for internal HTTPS  | <code>/etc/ssl/certs/selfsigned.crt`</code> | <code>/etc/nginx/ssl/selfsigned.crt`</code>   |
|                         |                              | <code>   `</code>                           | <code>/etc/ssl/private/selfsigned.key`</code> |

### ### Behavior

- **Bare IP (`192.168.2.10`)** → Redirects ``` to `prod.mitchellnet.local``, returns `404`` for `/prod`` and `/test``.
- **Subdomains:**
  - `prod.mitchellnet.local`` → `nginx-prod`` container (adds header `X-Upstream: prod``)
  - `test.mitchellnet.local`` → `nginx-test`` container (adds header `X-Upstream: test``)

---

## ## 📁 Directory Structure

```

```
/home/andrew/web_server/
├── docker-compose.yml
├── Makefile
├── includes/
│   ├── prod-env-banner.html
│   └── test-env-banner.html
├── html/
│   ├── prod/
│   └── test/
└── nginx/
```

```

├── conf.d/
│   ├── 000-bareip.conf
│   ├── prod.conf
│   ├── test.conf
│   └── proxy.conf.off
└── _snap_YYYY-MM-DD_HHMMSS/  # Auto snapshots
...

```

🌀 Daily Operations

Command	Description
`make check`	Tests redirects and subdomain routing
`make reload`	Reloads nginx-proxy configuration
`make logs`	Shows last 80 lines of nginx-proxy logs
`make snapshot`	Saves current configs under `nginx/_snap_`
`make restart-proxy`	Recreates nginx-proxy only
`make restart-backends`	Recreates both prod/test backend containers

🗺 Backup & Version Control

1 Copy this setup to your **development machine**

On the **production host** (`192.168.2.10`):

```

```bash
cd /home/andrew
tar -czvf web_server_backup_20251018.tar.gz web_server
scp web_server_backup_20251018.tar.gz andrew@dev-machine:/home/andrew/
...

```

On the **development machine**:

```

```bash
cd /home/andrew
tar -xzf web_server_backup_20251018.tar.gz
cd web_server
docker compose up -d
...

```

This allows testing or editing configs without overwriting production files.

2 Back up configuration to **GitHub**

From `/home/andrew/web\_server`:

```
```bash
make snapshot
cd nginx/_snap_YYYY-MM-DD_HHMMSS
tar -czvf nginx_config_backup.tar.gz conf.d docker-compose.yml
mv nginx_config_backup.tar.gz ../../
cd ../../
git add nginx_config_backup.tar.gz
git commit -m "Backup: nginx configuration snapshot 2025-10-18"
git push origin main
```
```

To **restore** later:

```
```bash
tar -xzf nginx_config_backup.tar.gz -C nginx/
docker compose up -d --force-recreate --no-deps nginx-proxy
```
```

📝 Notes

- Only internal use (`.local`) — not exposed externally.
- SSL is self-signed; browsers will show a warning (acceptable for local/internal).
- No dynamic proxy scripts or autoconf; everything is manual and snapshot-controlled.

```
**Author:** Andrew Mitchell — *TheAgingApprentice*
**System:** Ubuntu Server 24.04.2 LTS (iMac19,1)
**Project:** `MitchellNET` Internal Web Server
**Last Updated:** 2025-10-18 23:41:21
```

Dev Reverse Proxy Setup (Parallels Production)

This bundle gives you a drop-in development setup that mirrors production:

- TLS-terminating **nginx-proxy** routing by host:
 - `prod.mitchellnet.local` → `nginx-prod-dev` (serves `/html/prod`)
 - `test.mitchellnet.local` → `nginx-test-dev` (serves `/html/test`)
- Backends are plain `nginx:latest` containers serving static files
- Works with your repo layout: `html/prod` and `html/test`

1) Hosts entries (on your dev machine)

Edit `/etc/hosts` and add:

```
127.0.0.1 prod.mitchellnet.local
127.0.0.1 test.mitchellnet.local
```

...

2) Generate trusted dev certs

From the repo root (where this bundle lives):

...

make certs

...

This uses [mkcert](https://github.com/FiloSottile/mkcert) to create `./ssl/dev.crt` and `./ssl/dev.key`.

3) Start the stack

...

make up

...

Now open:

- https://prod.mitchellnet.local
- https://test.mitchellnet.local

4) Live editing

Edit files under `html/prod` or `html/test` and refresh the browser. No container reload is required for HTML/CSS/JS.

If you edit `proxy/default.conf`, run:

...

make reload

...

Notes

- Use **root-relative** URLs in your HTML and partials: `/css/style.css`, `/img/...`, `/tools/`, etc.
- Ensure your client-side include script pulls `/header.html` and `/footer.html` from each site root.
- The compose file name is `docker-compose.dev.yml` so it won't collide with your prod compose.